

micrograd

someonecantcode

March 15, 2026

Abstract

This is an explanation of my code, design choices, and how it's implementation is efficient. Additionally, we will also discuss potential future features and classes for new behaviors.

Contents

1	Beginnings	2
2	Back Propagation	3
2.1	Topological Sort	4
3	Draft: Creating a Program to read MNIST numbers	4
3.1	MNIST training loop	5

List of Theorems

1.1	DEFINITION (Neuron)	2
1.2	DEFINITION (Layer)	2
1.3	DEFINITION (MLP)	2
2.1	DEFINITION (Chain Rule)	3
2.2	DEFINITION (Gradient)	3
3.1	DEFINITION (Mean Square Loss)	5
3.2	DEFINITION (Cross Entropy Loss)	5



Figure 1: hehehaha cool mega mind

1 Beginnings

A neural network is composed of layers which are composed of neurons. This structure (with a non-linear function) is very useful as it allows to approximate anything and shape it to any behavior we want. It's basically linear regression but on steroids. We will forward pass inputs and receive an output. With the output, we will calculate the loss and run back propagation for the individual gradients and apply gradient descent on the parameters to reduce the loss value. In turn, this will result in a model that is better suited toward the result. Repeating this process will eventually lead the loss function reaching a minimum.

Let us define all of the internal structures needed towards a Multi Layer Perceptron (MLP).

DEFINITION 1.1 (Neuron). A neuron or perceptron is denoted by the following:

$$N = \sum_{i=0}^n w_i x_i + b, \quad n \text{ being the input size.} \quad (1)$$

$$N = [x_0 \quad x_1 \quad \cdots \quad x_i] \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_i \end{bmatrix} + [b], \quad i \text{ being the number of inputs.} \quad (2)$$

DEFINITION 1.2 (Layer). A layer of neurons or a single layer perceptron is denoted by the following:

$$L = \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_i \end{bmatrix}, \quad i \text{ being the amount of neurons.} \quad (3)$$

DEFINITION 1.3 (MLP). A multi layer perceptron has multiple layers and it is denoted by the following:

$$MLP = [L_0 \quad L_1 \quad \cdots \quad L_i] = \begin{bmatrix} \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_a \end{bmatrix} & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_b \end{bmatrix} & \cdots & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_c \end{bmatrix} \end{bmatrix}, \quad a, b, c \text{ being the size of each layer.} \quad (4)$$

Now that we have some infrastructure to work with, we have to show how forward pass would work. Imagine we have a 2 layer (1 input/middle, 1 output). For the input layer we shall take in two inputs and output 4. The output layer must take 4 and we will let it output 1.

This is how the math would work out.

$$X_0 = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}, L_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}, N_i = [x_0 \quad x_1] \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} + [b]$$

$$L_1 = [N_0], N_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}^\top \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} = [N_0 \quad N_1 \quad N_2 \quad N_3] \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} + [b]$$

2 Back Propagation

To actually train and get the wanted behavior from a MLP, we must obtain the direction of largest growth or smallest otherwise known as the gradient of the parameters. With the gradient, we can use that to move in a direction of lower loss. With a little bit of multi-variable calculus and chain rule, we can obtain all of the local derivatives and in turn, their gradients. Here is the math behind it.

The most rudimentary case is to imagine taking derivatives at the root.

$$\frac{\partial L}{\partial c} = \frac{\partial L}{\partial c} \overset{1}{\cancel{\frac{\partial L}{\partial L}}} = \frac{\partial L}{\partial c} \quad (5)$$

However, imagine if we have a function of two variables with one being a composite. In this example, it should become clear that we will always calculate the outer derivative and multiply it by the inner derivative. At the start, we will simply take it's derivative and multiply it by one.

DEFINITION 2.1 (Chain Rule). With that, back propagation is defined by the chain rule.

$$L(c, d) = c + d, \quad c(a) = a + b; \quad (6)$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial a}} \quad (7)$$

Accumulating gradients:

$$L(c, d) = c(a) + d(a), \quad c(a) = a + b, \quad d(a) = a + c \quad (8)$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial a} + \frac{\partial L}{\partial d} \frac{\partial d}{\partial a}} \quad (9)$$

DEFINITION 2.2 (Gradient). We can take this farther and define the gradient of the loss function.

$$\boxed{\nabla L = \left\langle \frac{\partial L}{\partial a_0}, \dots, \frac{\partial L}{\partial a_i} \right\rangle} \quad (10)$$

This is a very powerful tool and the next thing to tackle is how we order our function so that we can create an algorithm to start from the root and propagate back.

2.1 Topological Sort

With back propagation, we will need to be able to know what order to propagate. This is basically a job scheduling problem so topological sort will be the perfect algorithm for this. Topological sort works by applying a post-order DFS and reversing the list. This can easily be done using a recursive formula to completely search each branch. Reversing should be extremely simple with a library.

Algorithm 1 Topological Sort

Require: $n \geq 0$

Ensure: $y = x^n$

$y \leftarrow 1$

$X \leftarrow x$

$N \leftarrow n$

while $N \neq 0$ **do**

if N is even **then**

$X \leftarrow X \times X$

$N \leftarrow \frac{N}{2}$

▷ This is a comment

else if N is odd **then**

$y \leftarrow y \times X$

$N \leftarrow N - 1$

end if

end while

Algorithm 2 DFS for Topological Sort

procedure DFS(v , topo, visited)

if $v \notin$ visited **then**

 visited $\leftarrow$ visited $\cup \{v\}$

for all child $\in$ v .prev_children **do**

 DFS(child, topo, visited)

end for

 append(topo, v)

end if

end procedure

3 Draft: Creating a Program to read MNIST numbers

First, we initialize an MLP with an input size and layer params. The layer params are simply the size of each layer.

After that, we run a training loop. A training loop is simply composed of these fundamental steps: sequential bullet points.

Zero out all of the local gradients. Feedforward the input through the MLP and obtain an output. Calculate the loss with the output. Potentially reduce the loss with gradient descent.

Repeat with a new batch of data.

Time to break down each individual step. We must zero out all of the local gradients because we don't want any past calculations to interfere. Basically just a clean slate.

Feed forward is just running the input through the MLP. This is usually done in matrices or tensors.

With the output, we can calculate how far off we are and grade the model. Our goal for the MLP is to reduce the loss calculated. We can punish the model by having a larger loss. This brings us to loss functions that use this relationship of the magnitude of wrongness to the magnitude of loss. There is simple error loss, mean squared loss, and cross entropy loss.

Now we must reduce the loss as much as possible which is in the direction of the gradients. We run back propogration through each operaiton to obtain all the local derivatives. Think of the local derviatives as a ratio of how much each individual component helps toward the output. Almost like ingredients toward a dish. After, simply adjust the weights and biases in the direction of the gradient. If we want to increase the value, positive gradient and decrease the value, negative gradient. Comes in theory with the directional derivative and the dot product that has a cosine function.

3.1 MNIST training loop

In the case with the NNIST loop, we go through each component of the training loop one by one and build it. When buildinging, it's important to test each component instead of writing all at once. First we obtain the data as a 784 (28 x 28 image) vector and additionally normalize every value which is to divide by 255. We get this number because the data is in the form of brightness values from 0-255 hence the normalization factor. We additionally obtain the proper correct label to check in the loss function.

Now feed forward and obtain an output vector. I decided for an 10 size output vector for each number from 0-9. We run this softmax to gain a "percentage" distrubution of numbers and to clamp them from 0-1. This is import due to the one-hot labelling used for calculating the loss.

I implemented both mean squared and cross entropy loss. In comparison with the one-hot label, we can compare that with our 10 size output vector. Mean squared simply sums all of the the squared difference between each index and normalizing it with the size. (Hence the name mean square.) Cross entropy takes the sum of the expected value multiplied by the log of the predicted output value.

Given here are the algorithms.

DEFINITION 3.1 (Mean Square Loss).

$$\mathcal{L}(x) = \frac{\sum_i^n (\text{expected}_i - \text{predicted}_i)^2}{n} \quad (11)$$

DEFINITION 3.2 (Cross Entropy Loss).

$$\mathcal{L}(x) = \frac{\sum_i^n (\text{expected}_i \cdot \log(\text{softmax}(\text{predicted}_i)))^2}{n} \quad (12)$$

Run back propogration and now gradient descent.

$$\begin{aligned} D_{\hat{\mathbf{u}}} f &= \nabla f \cdot \hat{\mathbf{u}} \\ &= |\nabla f| |\hat{\mathbf{u}}| \cos(\theta), \quad -1 \leq \cos(\theta) \leq 1 \\ &\implies -\nabla f \leq D_{\hat{\mathbf{u}}} f \leq \nabla f \end{aligned} \quad (13)$$